

Markov TA Lab Current Project Guide

Date: 2026-05-17

This guide explains how to use the project in its current state, what has already been built, what each script and module does, and where the project is going next. It is written as a learning document, not just a command checklist, so it includes the reasoning behind the pieces.

1. Project Goal

The Markov TA Lab is a Python research workspace for asking one central question:

When price interacts with support, resistance, or trend structure, what is the probability of each next market state, and does that probability create positive expected value after costs?

The project is not a trading bot. It is not connected to a broker. It does not place trades. The goal is to build a repeatable research loop:

- download data
- clean data
- compute indicators
- detect support/resistance zones
- label market states
- estimate transition probabilities
- forecast future states
- measure expected value
- compare against baselines
- only then consider backtesting rules

The current project is through the visible Markov model phase. Expected value and backtesting come next.

2. Repository Location

The project is here:

C:\Users\CaveUser\Desktop\Project Project\markov-ta-lab

Start every terminal session from the repo root:

```
cd "C:\Users\CaveUser\Desktop\Project Project\markov-ta-lab"
```

The GitHub repo is:

<https://github.com/geraldfillman/markov-ta-lab>

3. Current Test Status

Run:

```
python -m pytest -q
```

Current expected result:

38 passed, 3 skipped

The skipped tests are expected. They belong to the future Backtest Agent phase.

The tests currently verify:

```
project setup
CLI/script ergonomics
data cleaning and IO
technical indicators
support/resistance levels
state labeling
Markov transition/forecast utilities
module import smoke tests
```

4. Environment Notes

The project was designed for Python 3.11, but the current code built so far can run under your existing Python environment too.

You created a .venv/ directory locally. That folder is ignored by git. It should never be committed.

If you use PowerShell to activate a venv, use:

```
& ".\venv\Scripts\Activate.ps1"
```

The & matters in PowerShell. It tells PowerShell to run the script path.

If package installs fail on Python 3.14, that is mostly because some later research packages, such as hmmlearn and ruptures, may need prebuilt wheels or Microsoft C++ Build Tools. Those packages are not needed for the current passing tests.

5. Main Script

The main runnable project script right now is:

```
python notebooks\01_data_download.py
```

Even though it lives in the notebooks/ folder, it is currently a normal Python script. It is not a .ipynb notebook yet.

When you run it, it:

1. Downloads OHLCV data from yfinance.

2. Saves raw CSV files.
3. Adds technical indicators.
4. Adds support/resistance features.
5. Adds deterministic state labels.
6. Saves enriched parquet files.
7. Prints a missing-data report.

The first experiment universe is:

SPY
QQQ
IWM
XLK
SMH
XLE
XLF
GLD
TLT

Raw files are saved here:

data/raw/*.csv

Processed files are saved here:

data/processed/*.parquet

Both raw and processed generated data files are ignored by git.

6. What yfinance Provides

yfinance provides the raw market candles:

Date
Open
High
Low
Close
Volume

This is enough for the first research system. The project computes the technical features itself. yfinance does not give us Markov states, support/resistance zones, ATR distances, expected value, or transition probabilities. Those are derived from the OHLCV data.

7. Current Pipeline Flow

The pipeline currently flows like this:

```
yfinance  
-> src.data.download_ohlc  
-> src.indicators.add_indicators  
-> src.levels.detect_levels
```

```
-> src.states.label_states
-> data/processed/*.parquet
-> src.markov can estimate probabilities from the state column
```

The Markov functions are implemented as library functions. They are not yet wired into the main data download script.

8. Module-by-Module Guide

8.1 'src/data.py'

Purpose:

Download, normalize, save, load, and report on OHLCV data.

Important functions:

```
download_ohlcvc(symbols, start, end)
save_raw(data)
save_processed(data)
load_processed(symbol)
missing_data_report(data)
```

The Data Agent normalizes downloaded data into:

```
Open
High
Low
Close
Volume
```

It sorts dates, drops rows missing required OHLCV values, saves raw CSV files, and saves processed parquet files.

8.2 'src/indicators.py'

Purpose:

Compute technical indicators from OHLCV data.

Currently computed:

```
sma_20
sma_50
sma_200
ema_20
atr_14
rsi_14
bb_width_20
return_1d
return_5d
return_10d
```

```
realized_vol_20
volume_zscore_20
dist_to_sma_20
dist_to_sma_50
dist_to_sma_200
```

Important idea:

All indicators are aligned to the current timestamp and use current or prior data only.

No indicator should use tomorrow's price to describe today's setup.

8.3 'src/levels.py'

Purpose:

Compute support and resistance zones using only prior data.

Current simple model:

```
nearest_resistance = prior rolling high
nearest_support = prior rolling low
```

The important guardrail is `.shift(1)`. That means today's high or low cannot become today's support/resistance.

Currently computed:

```
nearest_support
nearest_resistance
support_zone_low
support_zone_high
resistance_zone_low
resistance_zone_high
dist_to_support_atr
dist_to_resistance_atr
```

Zones are ATR-normalized. Support and resistance are not treated as exact single-price lines.

Example:

```
resistance = 749.53
ATR = 7.20
zone width = 0.5 * ATR = 3.60
resistance zone = 745.93 to 753.13
```

8.4 'src/states.py'

Purpose:

Label every bar into exactly one deterministic market state.

Current state map:

```
0  FAR_FROM_LEVEL
1  APPROACHING_SUPPORT
2  TOUCHING_SUPPORT
3  SUPPORT_RECLAIM
4  SUPPORT_BREAKDOWN
5  APPROACHING_RESISTANCE
6  COMPRESSION_BELOW_RESISTANCE
7  RESISTANCE_BREAKOUT
8  BREAKOUT_RETEST
9  FAILED_BREAKOUT
10 CONTINUATION
11 CHOP_OR_NO_EDGE
```

Important functions:

```
label_states(df)
state_frequency_report(states)
flag_rare_states(states)
```

The first state rules are intentionally simple and deterministic. They are not optimized yet. Their job is to create a clean, testable state sequence that can feed the Markov model.

Important design choice:

Every row gets exactly one state.

If a row is missing warm-up features, it becomes:

CHOP_OR_NO_EDGE

8.5 'src/markov.py'

Purpose:

Estimate transition probabilities from the state sequence.

Important functions:

```
estimate_transition_matrix(states, n_states, alpha=0.0)
forecast_state_probs(P, current_state, horizon)
stationary_distribution(P)
walkforward_forecasts(states, n_states, lookback, horizons=(1, 5, 10, 20))
```

The transition matrix answers:

Given the current state, how often did the next bar move into each possible state?

Example concept:

```
FAILED_BREAKOUT -> FAR_FROM_LEVEL: 42.66%
FAILED_BREAKOUT -> BREAKOUT_RETEST: 18.80%
FAILED_BREAKOUT -> FAILED_BREAKOUT: 14.63%
```

Multi-step forecasts use matrix powers:

```
P^5
P^10
P^20
```

Walk-forward forecasts estimate the matrix from only prior states. That avoids lookahead bias.

9. How To Inspect Processed Data

Run the pipeline:

```
python notebooks\01_data_download.py
```

Then inspect SPY:

```
python -c "import pandas as pd; df=pd.read_parquet('data/processed/SPY.parquet'); print(df.tail()[['Close', 'sma_20', 'atr_14', 'nearest_support', 'nearest_resistance', 'state']])"
```

To see state names:

```
python -c "import pandas as pd; from src.config import STATE_LABELS; df=pd.read_parquet('data/processed/SPY.parquet'); print({i: STATE_LABELS[i] for i in sorted(df['state'].dropna().astype(int).unique())})"
```

To inspect one date:

```
python -c "import pandas as pd; from src.config import STATE_LABELS; df=pd.read_parquet('data/processed/SPY.parquet'); dt=pd.Timestamp('2025-12-31'); row=df.loc[dt]; print(row[['Close', 'nearest_support', 'nearest_resistance', 'dist_to_support_atr', 'dist_to_resistance_atr', 'state']]); print('label:', STATE_LABELS[int(row['state'])])"
```

Note: the default DEFAULT_END is currently 2025-12-31, so the saved processed data from the script will not include May 2026 unless the date range is changed or a direct pull is run.

10. How To Run A Direct Markov Smoke Check

This command downloads SPY, computes indicators, levels, states, and a 5-step Markov forecast:

```
python -c "from src.config import N_STATES, STATE_LABELS; from src.data import download_ohlcv; from src.indicators import add_indicators; from src.levels import detect_levels; from src.states import label_states; from src.markov import estimate_transition_matrix, forecast_state_probs; data=download_ohlcv(['SPY'],
```

```
'2024-01-01', '2026-05-16'); frame=detect_levels(add_indicators(data['SPY']));
states=label_states(frame); P=estimate_transition_matrix(states, N_STATES,
alpha=1e-6); current=int(states.iloc[-1]); probs=forecast_state_probs(P, current, 5);
print('current_state', current, STATE_LABELS[current]);
print(sorted([(STATE_LABELS[i], round(float(p),4)) for i,p in enumerate(probs)],
key=lambda x: x[1], reverse=True)[:5])"
```

Recent smoke-check result:

```
current_state 9 FAILED_BREAKOUT
top 5-step forecast:
FAR_FROM_LEVEL          0.4266
BREAKOUT_RETEST         0.1880
FAILED_BREAKOUT         0.1463
APPROACHING_RESISTANCE  0.1203
COMPRESSION_BELOW_RESISTANCE 0.0729
```

This is not a trading signal yet. It is only a probability map from the current deterministic state model.

11. What Has Not Been Built Yet

Not built yet:

```
expected value tables
forward return evaluation
trade rules
backtesting
benchmark comparison
parameter sweeps
HMM regimes
change-point detection
volatility filters
macro filters
report generation
```

This is intentional. The project is being built in layers so we can verify each piece.

12. Current Git State

Recent completed phases:

```
data pipeline
technical indicators
support/resistance levels
state labeling
Markov transition and forecast utilities
```

Generated local files ignored by git:

```
.venv/
.pytest_cache/
.pytest_tmp/
```

```
.test_output/  
data/raw/*.csv  
data/processed/*.parquet
```

13. How To Think About The Current Model

Right now the system can answer:

```
What state is each historical bar in?  
How often does each state transition to another state?  
Given today's state, what are the 1-step and multi-step future state probabilities?
```

It cannot yet answer:

```
Is this setup profitable?  
Does it beat a simple breakout rule?  
What is the expected return after costs?  
What is the drawdown profile?  
Should a simulated strategy enter or exit?
```

Those questions belong to the next two phases: Expected Value Agent and Backtest Agent.

14. Roadmap For Tomorrow

Target date: 2026-05-18

Phase 1: Expected Value Agent

Primary file:

```
src/metrics.py
```

Tests:

```
tests/test_metrics.py
```

Build:

```
forward_return_5  
forward_return_10  
forward_return_20  
average forward return by state  
win rate by state  
average win  
average loss  
expected value after costs  
sample count by state  
rare-state warnings
```

The important question:

```
Which states historically had positive average forward returns after estimated costs?
```

Phase 2: First State Expectancy Table

Create a table like:

```
state
label
count
avg_forward_return_5
avg_forward_return_10
win_rate_5
avg_win_5
avg_loss_5
ev_after_cost_5
ev_after_cost_10
```

Save to:

reports/tables/

Phase 3: Markov Plus Expected Value

Combine:

```
current state
forecast probabilities
historical forward return by destination/current state
expected value after costs
```

This begins to answer:

If the model thinks continuation is more likely, does that actually imply positive expected value?

Phase 4: First Human-Readable Experiment Report

Create a first report in:

reports/runs/

Include:

```
question
hypothesis
data used
state definitions
transition matrix summary
state expectancy table
what worked
what failed
bias checks
next experiment
```

Phase 5: Prepare Backtest Agent

Only after Expected Value Agent is working, start Backtest Agent.

The first backtest should be narrow:

- long-only
- daily bars
- ETF universe
- fixed 5-bar or 10-bar exit
- costs included
- compare to simple breakout baseline

Do not add HMMs, macro filters, or change-point detection tomorrow unless the simple state and expected value analysis is working.

15. Questions To Bring Tomorrow

Good questions to ask after reading:

- Do the current state definitions feel intuitive?
- Should approaching resistance mean within 1 ATR or 1.5 ATR?
- Should failed breakout memory be 3, 5, or 10 bars?
- Should support/resistance use 60, 126, or 252 bars?
- Should state labels be asset-specific or estimated across all ETFs together?
- Should expected value be measured by current state only, or current state plus forecast destination?
- What transaction cost assumption should we use first?

These choices matter, but we should test them systematically rather than tune them by feel.